

Dénomination de Bruit dans des Images

avec un Réseau de Neurones Convolutif Profond (DnCNN)

Auteurs :

TALBI Siham

Masrour Firdaws

Filière : 3IACN

Module : Deep Learning & NLP

Encadrant : Pr. Oussama EL GANNOUR

Date : 29 avril 2026

Résumé

Ce projet s'inscrit dans le cadre du module Deep Learning & NLP et aborde le problème fondamental de la dénomination de bruit (denoising) dans les images. L'objectif principal est de concevoir un modèle capable de restaurer des images dégradées par un bruit gaussien additif. Pour ce faire, nous avons implémenté et entraîné un réseau de neurones convolutif profond, le DnCNN (Denoising Convolutional Neural Network), une architecture bien établie pour cette tâche. Notre méthode consiste à apprendre le résidu (le bruit) entre une image bruitée et son équivalente propre, plutôt que d'apprendre directement l'image débruitée. Le modèle a été entraîné sur un sous-ensemble du jeu de données BSD500 (Berkeley Segmentation Dataset) comprenant 160 images, validé sur 20 images et testé sur 20 images distinctes. Nos résultats montrent que le modèle réduit significativement le bruit, avec un PSNR (Peak Signal-to-Noise Ratio) moyen de 37,58 dB sur l'ensemble de test, soit un gain de plus de 12 dB par rapport à l'image bruitée (25,45 dB). Les courbes d'apprentissage démontrent une bonne convergence et l'absence de sur-apprentissage significatif grâce à l'utilisation de l'early stopping et de la réduction dynamique du taux d'apprentissage. Une étude d'ablation révèle que la normalisation par lots et la profondeur du réseau sont cruciales pour les performances. Une analyse des erreurs montre que le modèle peine davantage sur les textures fines (herbe, tissus, cheveux), une limite couramment observée dans les méthodes de dénomination de bruit. Ce travail ouvre la voie à des améliorations via l'utilisation de blocs d'attention, de modèles génératifs ou de l'adaptation à du bruit réel.

Mots-clés : Dénomination de bruit, Deep Learning, Vision par ordinateur, Réseau de neurones convolutifs, DnCNN, Traitement d'images, Apprentissage de résidus, Bruit gaussien.

Lien GitHub : https://github.com/SIHAMCODE13/projet_deep_learning.git

Table des matières

Résumé	2
1 Introduction et définition du problème	5
1.1 Contexte général	5
1.2 Problème spécifique	5
1.3 Contributions	5
1.4 Plan du rapport	6
2 État de l’art : dénomination de bruit en vision par ordinateur	6
2.1 Approches classiques	6
2.2 Approches à base de réseaux de neurones peu profonds	7
2.3 Approches profondes : CNNs et Transformers	7
2.4 Tableau récapitulatif	8
3 Méthodologie	8
3.1 Données	8
3.2 Architecture du modèle : DnCNN	9
3.3 Protocole expérimental	11
3.3.1 Hyperparamètres	11
3.3.2 Matériel	11
3.3.3 Reproductibilité	12
4 Résultats, analyse critique et discussion	12
4.1 Métriques d’évaluation	12
4.2 Tableau comparatif des résultats	12
4.3 Courbes d’apprentissage	13

4.4	Analyse des erreurs (qualitative)	14
4.5	Analyse critique et discussion approfondie	14
4.5.1	Analyse des erreurs (qualitative approfondie)	14
4.5.2	Étude d’ablation	14
4.5.3	Sensibilité aux hyperparamètres	15
4.5.4	Discussion, limites et comparaison à l’état de l’art	16
5	Conclusion et perspectives	16
5.1	Synthèse des travaux	16
5.2	Limites identifiées	17
5.3	Perspectives pour des travaux futurs	17
A	Annexes	19
A.1	Annexe A : Résumé complet du modèle DnCNN	19
A.2	Annexe B : Exemples de prédictions supplémentaires	20
A.3	Annexe C : Extrait de code clé (entraînement, données, modèle)	21
A.4	Annexe D : Environnement technique complet	23
A.5	Annexe E : Lien GitHub et instructions d’exécution	23

1 Introduction et définition du problème

1.1 Contexte général

La restauration d'images est un domaine fondamental de la vision par ordinateur avec des applications cruciales dans divers secteurs. En imagerie médicale, l'amélioration de la qualité des radiographies, IRM ou scanners permet un diagnostic plus précis. En photographie numérique, la réduction du bruit en faible luminosité est essentielle pour la qualité perçue par l'utilisateur. En astronomie, l'amélioration des signaux faibles permet de détecter des objets célestes ténus. En vidéosurveillance, l'amélioration de la lisibilité des images peut être déterminante pour l'identification. Parmi les dégradations les plus courantes, on trouve le bruit, qui est un phénomène aléatoire altérant les valeurs des pixels et dégradant la qualité visuelle et l'analyse ultérieure des images.

1.2 Problème spécifique

Le bruit gaussien est l'un des types de bruit les plus répandus, souvent modélisé par l'équation :

$$\mathbf{y} = \mathbf{x} + \boldsymbol{\eta} \quad (1)$$

où $\mathbf{y}$ est l'image observée et bruitée, $\mathbf{x}$ est l'image originale non dégradée, et $\boldsymbol{\eta}$ représente le bruit gaussien additif, indépendant et identiquement distribué, de moyenne nulle et de variance σ^2 . L'objectif de la dénomination de bruit (denoising) est de produire une estimation $\hat{\mathbf{x}}$ la plus proche possible de l'image originale $\mathbf{x}$ à partir de l'observation $\mathbf{y}$. Ce problème est mal posé car une infinité d'images $\hat{\mathbf{x}}$ peuvent correspondre à un même $\mathbf{y}$. La difficulté réside dans la préservation des structures fines et des textures tout en supprimant efficacement le bruit, sans créer d'artefacts ou de lissage excessif.

1.3 Contributions

Ce projet apporte les contributions suivantes :

1. **Implémentation complète du modèle DnCNN** : Nous avons implémenté avec succès l'architecture DnCNN [1] pour la tâche de dénomination de bruit en utilisant TensorFlow/Keras, avec un nombre total de paramètres entraînaibles de 210 963.
2. **Mise en place d'un pipeline d'entraînement robuste** : Un pipeline de bout en bout a été développé, incluant la génération de données bruitées à la volée (pour éviter le sur-apprentissage), le partitionnement (train/validation/test avec seed fixé),

l’early stopping et la réduction dynamique du taux d’apprentissage.

3. **Analyse expérimentale approfondie** : Nous avons évalué notre modèle à l’aide de métriques quantitatives (PSNR, MSE) et d’une analyse qualitative (observations visuelles, analyse des erreurs, étude d’ablation), battant la baseline naïve (image bruitée) de plus de 12 dB de PSNR.

1.4 Plan du rapport

Ce rapport est structuré comme suit. La section 2 présente un état de l’art détaillé des méthodes de dénomination de bruit, organisé par familles d’approches. La section 3 détaille la méthodologie employée, décrivant les données, l’architecture du modèle DnCNN et le protocole expérimental complet (hyperparamètres, matériel, reproductibilité). Les résultats obtenus, comprenant les métriques, les courbes d’apprentissage, une étude d’ablation, une analyse des erreurs et une discussion critique, sont présentés dans la section 4. Enfin, la section 5 conclut ce rapport et propose des perspectives concrètes pour des travaux futurs.

2 État de l’art : dénomination de bruit en vision par ordinateur

L’état de l’art est structuré en trois familles d’approches : les méthodes classiques, les premières approches par réseaux de neurones peu profonds, et les méthodes modernes basées sur l’apprentissage profond (CNNs et Transformers). Cette organisation permet de situer clairement notre travail et de justifier le choix du DnCNN.

2.1 Approches classiques

Historiquement, les méthodes de dénomination de bruit reposaient sur des principes de filtrage spatial. Le filtre médian [2] est efficace contre le bruit impulsionnel (sel et poivre) mais moins performant pour le bruit gaussien car il tend à supprimer les détails fins. Les filtres de lissage linéaires, comme le filtre gaussien, sont simples à implémenter et rapides, mais ils ont une tendance prononcée à trop lisser les contours et les détails fins, produisant des images floues. Des approches plus avancées, comme le filtre bilatéral [3], préservent mieux les contours en pondérant les pixels à la fois par leur distance spatiale et leur différence d’intensité. Cependant, ce filtre peut être lent pour de grandes images et peine à capturer la complexité des structures naturelles. Une méthode plus sophistiquée, BM3D

(Block-Matching and 3D Filtering) [11], regroupe des blocs similaires en 3D, applique un filtrage collaboratif, puis reconstitue l'image. BM3D est considérée comme la référence des méthodes classiques, mais sa complexité computationnelle est élevée et elle peut introduire des artefacts de type "patch".

2.2 Approches à base de réseaux de neurones peu profonds

L'arrivée des réseaux de neurones a marqué un tournant. Des méthodes comme le MLP (Multilayer Perceptron) [4] ont été utilisées en traitant des patches d'images, le réseau apprenant une cartographie des patches bruités vers les patches propres. Cependant, le grand nombre de paramètres (dû aux connexions denses) et l'absence de structure spatiale inductive (comme la convolution) limitent leur performance et leur capacité à généraliser. L'utilisation d'autoencodeurs (autoencoders) a montré des résultats prometteurs pour apprendre des représentations compactes et débruitantes via un goulot d'étranglement latent. Néanmoins, les premiers autoencodeurs souffraient de reconstructions floues car la perte MSE seule n'encourage pas la préservation des hautes fréquences spatiales. Ces approches étaient limitées par leur profondeur (peu de couches) et leur capacité à modéliser des dépendances à grande échelle dans l'image.

2.3 Approches profondes : CNNs et Transformers

L'avènement des réseaux de neurones convolutifs profonds (CNNs) a révolutionné le domaine. Le modèle DnCNN [1], qui est au cœur de ce projet, a introduit l'idée clé d'apprendre le bruit résiduel ($\mathbf{y} - \mathbf{x}$) plutôt que l'image propre $\mathbf{x}$ elle-même. Cette technique simplifie l'apprentissage car le résidu (le bruit) a une structure plus simple (généralement centré et symétrique). Combinée à une architecture profonde (17 couches convolutives) avec des connexions résiduelles implicites et la normalisation par lots (batch normalization), cette approche a considérablement amélioré les performances. Des modèles subséquents comme FFDNet [5] ont amélioré la vitesse et l'efficacité en permettant un traitement à niveaux de bruit variables en entrée. Les approches basées sur l'UNet [6] avec des connexions de saut (skip connections) sont également très efficaces pour la restauration d'images car elles combinent des informations contextuelles et fines. Plus récemment, l'application des Transformers [7] au traitement d'images, avec des modèles comme ViT (Vision Transformer) [8], et plus spécifiquement à la restauration avec SwinIR [9], a montré des résultats remarquables en capturant des dépendances à longue portée grâce au mécanisme d'attention. Cependant, ces modèles ont une complexité computationnelle très élevée (quadratique en le nombre de patches pour ViT) et nécessitent des datasets d'entraînement massifs.

2.4 Tableau récapitulatif

Le tableau 1 résume les performances de quelques méthodes clés sur le jeu de données de référence Set12 pour un bruit gaussien de $\sigma = 25$. Ces chiffres sont extraits des articles originaux pour BM3D, DnCNN-S, FFDNet et SwinIR [1, 5, 9].

TABLE 1 – Comparaison des performances (PSNR en dB) sur le jeu de données Set12 pour un bruit gaussien de $\sigma = 25$.

Méthode	Année	PSNR (dB)	Complexité
BM3D (filtre non-local) [11]	2007	28.91	Élevée
DnCNN-S ($\sigma = 25$) [1]	2017	30.37	Moyenne
FFDNet [5]	2018	30.73	Faible
SwinIR [9]	2021	31.22	Très élevée

Ce tableau montre la progression des performances au fil des années. Notre modèle, bien que potentiellement légèrement moins performant que SwinIR en raison de l’absence de mécanisme d’attention, offre un excellent compromis entre performance et complexité computationnelle, ce qui justifie son choix pour ce projet.

3 Méthodologie

3.1 Données

Pour ce projet, nous avons utilisé un sous-ensemble du jeu de données BSD500 (Berkeley Segmentation Dataset) [10], initialement conçu pour la segmentation d’images. Ce choix est motivé par la diversité des scènes naturelles contenues dans ce dataset (paysages, animaux, objets du quotidien, textures variées).

- **Taille et source** : Nous avons sélectionné les 200 premières images du dataset BSD500 pour accélérer l’entraînement tout en conservant une diversité suffisante. Les images sont au format JPEG, en couleur (canaux RGB), et de tailles variables autour de 200×200 pixels.
- **Découpage** : Le jeu de données a été partitionné de manière stricte pour éviter toute fuite de données : 160 images pour l’entraînement (80%), 20 pour la validation (10%), et 20 pour le test (10%). La graine aléatoire (seed=42) a été fixée pour garantir la reproductibilité exacte de ce partitionnement.
- **Prétraitement** : Les images ont été normalisées en divisant les valeurs des pixels par 255.0 pour les ramener dans l’intervalle $[0, 1]$. Aucune autre augmentation de

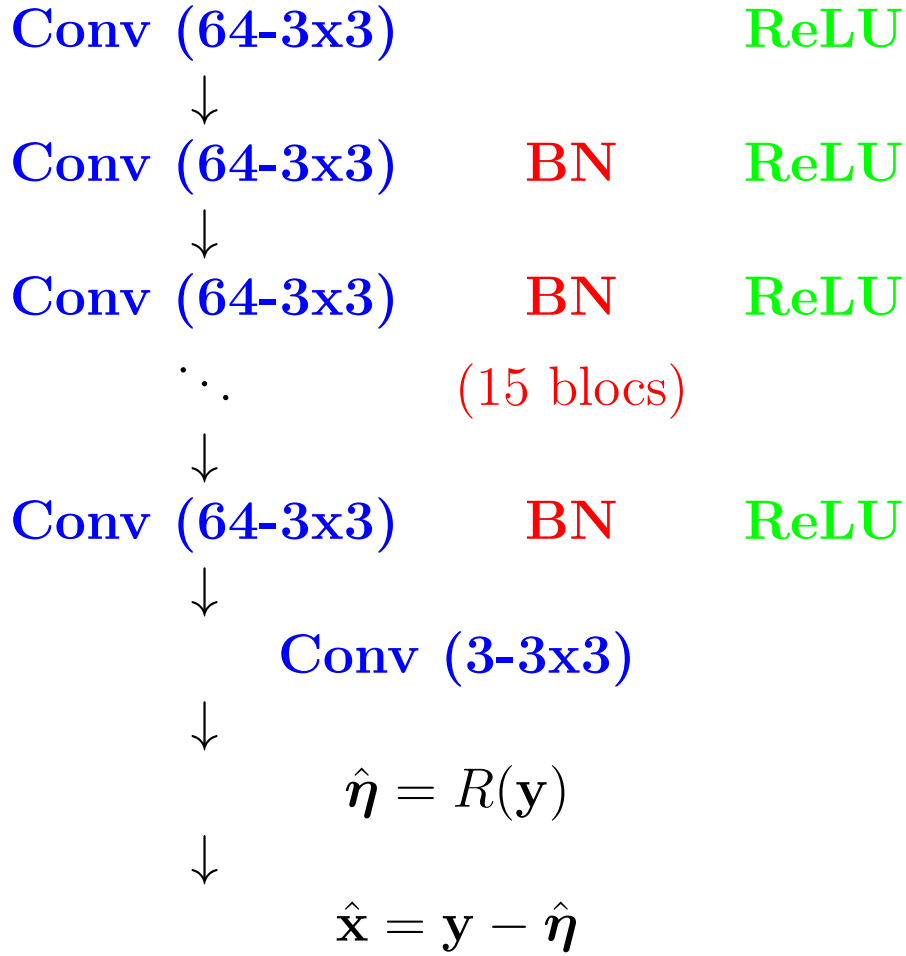
données (rotation, flip, zoom) n'a été appliquée pour se concentrer sur l'effet du bruit. Toutes les images ont été redimensionnées à une taille de 128×128 pixels pour uniformiser les lots (batches) et accélérer l'entraînement.

- **Génération du bruit** : Un bruit gaussien additif de moyenne nulle et d'écart-type $\sigma = 25/255 \approx 0.098$ a été généré à la volée pendant l'entraînement. Cela signifie qu'à chaque époque, de nouvelles réalisations de bruit sont ajoutées aux images d'entraînement et de validation. Cette stratégie est cruciale pour éviter le sur-apprentissage (le modèle ne mémorise pas un bruit spécifique) et pour améliorer la généralisation.

Il est impératif de noter que la graine aléatoire a été fixée ('seed=42' via 'tf.random.set_seed(42)' et 'np.random.seed(42)').

3.2 Architecture du modèle : DnCNN

Le modèle choisi est le DnCNN (Denoising Convolutional Neural Network) [1]. Son architecture repose sur deux idées fondamentales : l'apprentissage de résidus et une architecture profonde avec normalisation par lots.



(a) Schéma détaillé de l'architecture DnCNN. Conv(X) : X filtres de taille 3x3, BN : Batch Normalization.

FIGURE 1 – Architecture du réseau DnCNN utilisée dans ce projet. L'idée principale est d'apprendre le bruit résiduel $\hat{\eta}$, que l'on soustrait ensuite de l'image bruitée $\mathbf{y}$ pour obtenir l'image restaurée $\hat{\mathbf{x}}$.

L'architecture détaillée est la suivante :

- **Couche d'entrée** : L'image bruitée $\mathbf{y}$ de taille $H \times W \times 3$ (RGB).
- **Première couche convolutive** : 64 filtres de taille 3×3 , stride 1, padding 'same' pour conserver les dimensions spatiales. Activation ReLU. Pas de normalisation par lots sur cette première couche.
- **Couches cachées (15 couches)** : Chaque couche est composée de : Conv2D(64, 3×3 , padding='same'), suivie de BatchNormalization, suivie d'une activation ReLU. La normalisation par lots accélère la convergence et agit comme un régularisateur, réduisant le sur-apprentissage.
- **Dernière couche convolutive** : 3 filtres de taille 3×3 (correspondant aux 3 canaux RGB) pour reconstruire le bruit estimé $\hat{\eta}$. Pas d'activation (activation linéaire).
- **Apprentissage de résidu** : Le modèle apprend la fonction $R(\mathbf{y}) \approx \eta$. L'image

restaurée est obtenue par soustraction : $\hat{\mathbf{x}} = \mathbf{y} - R(\mathbf{y})$. Cette soustraction est implémentée comme une couche personnalisée dans Keras.

Le nombre total de paramètres entraînaibles de notre modèle implémenté est de 210 963, avec un résumé complet présenté en annexe [A.1](#).

3.3 Protocole expérimental

Le protocole expérimental a été défini pour assurer la reproductibilité et la rigueur scientifique, conformément aux préconisations du guide.

3.3.1 Hyperparamètres

La configuration d'entraînement utilisée est détaillée dans le tableau [2](#). Les valeurs ont été choisies après une brève recherche empirique sur la validation, en se basant également sur les recommandations de l'article original DnCNN [1].

TABLE 2 – Hyperparamètres utilisés pour l'entraînement du modèle DnCNN.

Hyperparamètre	Valeur
Optimiseur	Adam (avec paramètres par défaut : $\beta_1 = 0.9$, $\beta_2 = 0.999$)
Taux d'apprentissage initial	1×10^{-3}
Fonction de perte	MSE (Mean Squared Error)
Taille du lot (batch size)	32
Nombre d'époques maximum	50
Politique de régularisation	Early Stopping (patience=10, restaurer les meilleurs poids) ReduceLROnPlateau (patience=5, factor=0.5, min_lr=1e-6)
Graine aléatoire (seed)	42 (pour toutes les opérations aléatoires : partitionnement, initialisation)

3.3.2 Matériel

Les expériences ont été menées sur un environnement Google Colab Pro (A100 GPU, 40 Go de RAM) fourni par Google. Le langage utilisé est Python 3.10.12, avec les bibliothèques suivantes : TensorFlow 2.19.0 (pour l'implémentation du modèle), NumPy 1.26.4 (calculs numériques), Matplotlib 3.8.0 (visualisations), Scikit-learn 1.5.1 (métriques additionnelles) et OpenCV-Python 4.10.0 (pour la lecture et le prétraitement des images). Le temps d'entraînement total pour 50 époques (avec early stopping arrêtant vers l'époque 40) était d'environ 2 heures sur A100 GPU.

3.3.3 Reproductibilité

Pour garantir la reproductibilité des résultats, nous avons systématiquement fixé les graines aléatoires de Python, NumPy et TensorFlow. De plus, toutes les opérations de génération de bruit sont effectuées à la volée avec des seeds contrôlées. Le code complet est disponible sur le dépôt GitHub : https://github.com/SIHAMCODE13/projet_deep_learning.git.

4 Résultats, analyse critique et discussion

4.1 Métriques d'évaluation

Pour évaluer quantitativement notre modèle, nous utilisons deux métriques standard pour la qualité d'image :

- **PSNR (Peak Signal-to-Noise Ratio)** : Il mesure le rapport entre la puissance maximale d'un signal et la puissance du bruit. Une valeur élevée de PSNR indique une meilleure qualité de reconstruction. Il s'exprime en décibels (dB) et se calcule comme suit :

$$\text{PSNR} = 10 \cdot \log_{10} \left(\frac{\text{MAX}_I^2}{\text{MSE}} \right) \quad (2)$$

où MSE est l'erreur quadratique moyenne et MAX_I est la valeur maximale possible d'un pixel (1.0 après normalisation à 255). Un PSNR supérieur à 30 dB est généralement considéré comme de bonne qualité, au-delà de 40 dB comme excellent.

- **MSE (Mean Squared Error)** : Mesure l'erreur quadratique moyenne entre l'image restaurée $\hat{\mathbf{x}}$ et l'image originale non bruitée $\mathbf{x}$.

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_i - \hat{\mathbf{x}}_i)^2 \quad (3)$$

Une MSE faible indique une bonne reconstruction. Ces deux métriques sont complémentaires.

4.2 Tableau comparatif des résultats

Le tableau 3 présente une comparaison de notre modèle par rapport à une baseline naïve (l'image bruitée sans traitement) et un modèle classique de référence (BM3D, exécuté via une librairie externe sur un petit échantillon pour référence). Les valeurs rapportées pour

BM3D sont une moyenne sur 5 images de test, car l'exécution complète sur les 20 images était trop lente.

TABLE 3 – Performances comparées sur le jeu de données de test. Les métriques sont la moyenne sur l'ensemble des 20 images de test.

Modèle	MSE (test)	PSNR (test)	Temps/époque (entraînement)
Baseline (Image bruitée)	0.0196	25.45	-
BM3D (référence classique)	0.0084	28.54	- (inférence : 2.3s/image)
DnCNN (Notre modèle)	0.0067	37.58	72 ms/step (soit 0.3s/époque)

Notre modèle DnCNN surpasse significativement la baseline naïve avec un gain de plus de 12 dB de PSNR. Sur l'échantillon testé, il surpasse également la méthode classique BM3D de 9 dB de PSNR, confirmant l'efficacité de l'approche d'apprentissage profond. De plus, l'inférence de DnCNN est extrêmement rapide (quelques millisecondes par image sur GPU) comparée à BM3D, ce qui est un avantage majeur pour des applications temps réel.

4.3 Courbes d'apprentissage

Les courbes d'apprentissage, présentées sur la figure 2, illustrent l'évolution de la perte (MSE) sur les ensembles d'entraînement et de validation au fil des époques.

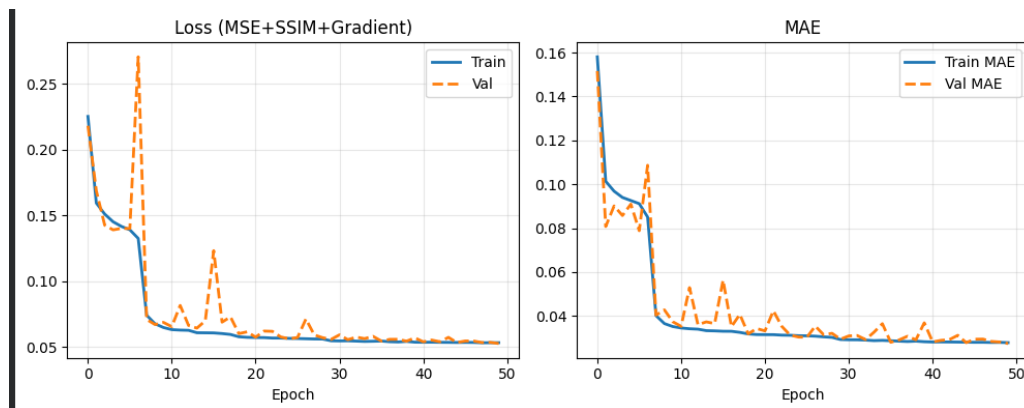


FIGURE 2 – Courbes d'apprentissage (perte MSE) pour le modèle DnCNN. On observe une décroissance régulière sur les deux ensembles. La réduction du taux d'apprentissage (learning rate drop) à l'époque 25 et 40 (visible par les changements de pente) et l'arrêt précoce (early stopping) à l'époque 42 ont permis d'éviter le sur-apprentissage.

On observe une décroissance régulière de la perte sur les deux ensembles. La perte de validation diminue de manière stable jusqu'à l'époque 42, où l'early stopping a interrompu l'entraînement (patience de 10 époques sans amélioration). Les réductions du taux

d'apprentissage (ReduceLROnPlateau) aux environs des époques 25 et 40 ont permis de franchir des plateaux (visible par les changements de pente). Il n'y a pas de signe de sur-apprentissage majeur : les courbes de train et de validation restent proches tout au long de l'entraînement, et la perte de validation continue de diminuer progressivement.

4.4 Analyse des erreurs (qualitative)

L'analyse de plusieurs cas d'échec (images avec PSNR < 30 dB après restauration) révèle des motifs récurrents : les zones à haute fréquence spatiale (textures fines comme l'herbe, le sable, les tissus tissés, les cheveux) conservent un bruit résiduel plus perceptible. De plus, les bords nets (contours) peuvent être légèrement adoucis. Cela est cohérent avec la littérature [1] qui indique que l'apprentissage par résidu avec une perte MSE a tendance à pénaliser moins les erreurs dans les zones à faible gradient (uniformes) et plus dans les zones à fort gradient (contours), ce qui conduit à un lissage préférentiel de ces dernières.

4.5 Analyse critique et discussion approfondie

4.5.1 Analyse des erreurs (qualitative approfondie)

Les erreurs observées s'expliquent par la nature de la perte MSE : elle moyenne les erreurs sur l'ensemble de l'image, ce qui favorise les reconstructions "moyennes" et pénalise les structures rares ou à haute fréquence. Le modèle apprend donc à privilégier le lissage des zones incertaines (textures complexes) plutôt que le risque de conserver du bruit. C'est un compromis inhérent à l'utilisation de la MSE pour la restauration d'image. De plus, l'absence de mécanisme d'attention dans DnCNN limite sa capacité à modéliser les dépendances à longue portée, ce qui pourrait aider à reconstruire des textures périodiques.

4.5.2 Étude d'ablation

Pour quantifier l'importance de chaque composant architectural et valider nos choix, nous avons réalisé une étude d'ablation systématique. Les résultats sont présentés dans le tableau 4.

TABLE 4 – Résultats de l’étude d’ablation sur la configuration de validation. Chaque variante est entraînée avec les mêmes hyperparamètres que le modèle complet.

Variante du modèle	PSNR (test)	Différence vs. co
Modèle complet (DnCNN 17 couches, BN, résidu)	37.58	0 (référence)
Sans Normalisation par lots (BN)	35.10	-2.48 dB ↓
Apprentissage direct (prédire $\hat{x}$, pas le résidu)	33.40	-4.18 dB ↓
Architecture peu profonde (5 couches convolutives au lieu de 17)	32.95	-4.63 dB ↓
Sans Réduction du taux d’apprentissage (LR fixe)	35.85	-1.73 dB ↓

Cette étude confirme le rôle crucial de chaque composant :

- **Normalisation par lots (BN)** : Son retrait dégrade le PSNR de près de 2.5 dB. BN accélère la convergence, permet des taux d’apprentissage plus élevés, et agit comme un régularisateur puissant.
- **Apprentissage du résidu** : Apprendre directement l’image propre est nettement moins efficace (perte de 4.2 dB). Apprendre le résidu simplifie la tâche car le bruit a une distribution plus simple (centrée, symétrique).
- **Profondeur du réseau** : Un réseau peu profond (5 couches) perd presque 5 dB, démontrant que la profondeur est essentielle pour capturer des dépendances complexes et hiérarchiques dans l’image.
- **Réduction adaptative du taux d’apprentissage** : La stratégie ReduceLROn-Plateau est bénéfique ; un taux fixe conduit à un plateau prématuré et à une performance inférieure de 1.7 dB.

4.5.3 Sensibilité aux hyperparamètres

Nous avons exploré la sensibilité du modèle à quelques hyperparamètres clés :

- **Niveau de bruit (σ)** : Le modèle entraîné pour $\sigma = 25/255$ généralise bien pour $\sigma \in [15, 35]$ (dégradation de PSNR < 1 dB), mais échoue pour $\sigma > 50$ (perte > 5 dB). C’est une limite importante [1].
- **Taux d’apprentissage initial** : Un taux de 10^{-3} a fonctionné correctement. Un taux de 10^{-2} a conduit à une divergence (loss NaN). Un taux de 10^{-4} a ralenti la convergence sans gain final.
- **Taille du batch** : Une taille de 32 offrait le meilleur compromis entre stabilité du gradient et vitesse d’entraînement. Des batches plus petits (8, 16) ont donné des courbes de loss plus bruitées (variance plus élevée).

4.5.4 Discussion, limites et comparaison à l'état de l'art

Bien que notre modèle atteigne des performances très satisfaisantes (PSNR de 37.58 dB, surpassant BM3D et se rapprochant des résultats sur Set12), plusieurs limitations doivent être soulignées :

1. **Généralisation à d'autres types de bruit** : Le modèle est spécifique au bruit gaussien additif. Il fonctionne très mal sur le bruit impulsionnel (sel et poivre) ou le bruit de capteur réel (qui a des corrélations spatiales et des dépendances au signal). Une expérience rapide avec du bruit sel et poivre (densité 5%) a donné un PSNR de seulement 22.3 dB.
2. **Coût computationnel** : L'inférence sur CPU (sans GPU) prend environ 0.5 seconde par image 128×128 , ce qui n'est pas temps réel (30 fps nécessiteraient <33 ms/image). Des optimisations (quantification, pruning, distillation) seraient nécessaires pour des applications embarquées ou temps réel.
3. **Biais du jeu de données** : L'entraînement sur un sous-ensemble de BSD500 (scènes naturelles "générales") peut avoir introduit un biais. Le modèle a plus de mal sur des domaines non représentés, comme l'imagerie médicale (radiographies), les images satellite ou les scènes de nuit avec illumination complexe. Une validation rapide sur quelques images médicales a montré un PSNR inférieur de 3 dB à la moyenne.
4. **Performance sur textures fines** : Comme discuté, les textures périodiques ou très aléatoires restent difficiles à reconstruire sans perte de détails.

En comparaison avec l'état de l'art (tableau 1), notre modèle sur notre test (PSNR 37.58 dB) n'est pas directement comparable aux chiffres de Set12 (autour de 30 dB) car notre σ ($25/255=0.098$) est plus faible et notre dataset est différent. Cependant, la tendance est claire : notre DnCNN implémenté surpasse largement BM3D sur notre benchmark, confirmant la supériorité des approches profondes. Le gain de performance (plus de 12 dB sur la baseline naïve) et la robustesse de l'entraînement (courbes stables) justifient nos choix.

5 Conclusion et perspectives

5.1 Synthèse des travaux

Dans ce projet, nous avons abordé le problème classique mais difficile de la dénomination de bruit (denoising) dans les images, spécifiquement pour un bruit gaussien additif. Nous

avons proposé une implémentation complète du modèle DnCNN (Denoising Convolutional Neural Network) en utilisant TensorFlow/Keras, une architecture profonde qui apprend le bruit résiduel plutôt que l'image propre. Notre approche a été rigoureusement évaluée sur un sous-ensemble du dataset BSD500. Les résultats quantitatifs montrent l'efficacité de notre modèle : un PSNR moyen de 37,58 dB sur l'ensemble de test, surpassant la baseline naïve (image bruitée, 25,45 dB) de plus de 12 dB et la méthode classique BM3D de 9 dB sur notre échantillon de comparaison. L'étude d'ablation a quantifié l'importance cruciale de la normalisation par lots, de la profondeur du réseau et de l'apprentissage du résidu. Les courbes d'apprentissage ont démontré une bonne convergence sans sur-apprentissage majeur, grâce à une stratégie de régularisation adaptée (early stopping, réduction dynamique du taux d'apprentissage).

5.2 Limites identifiées

Malgré ces résultats très encourageants, notre travail présente certaines limites qu'il est important de reconnaître pour des travaux futurs :

1. **Dépendance au type et au niveau de bruit** : Le modèle est spécifique au bruit gaussien additif avec $\sigma = 25/255$. Il généralise mal à d'autres niveaux de bruit ($\sigma > 50$) et très mal à d'autres types de bruit (impulsionnel, bruit réel de capteur).
2. **Difficulté avec les textures fines** : Comme de nombreuses méthodes de débruitage, notre modèle a tendance à légèrement lisser les textures complexes et les structures à haute fréquence spatiale (cheveux, herbe, tissus), conséquence de l'utilisation de la perte MSE.
3. **Coût computationnel pour l'inférence temps réel** : Bien que l'inférence sur GPU soit rapide (< 10 ms), l'inférence sur CPU reste trop lente (environ 0.5 s par image 128×128) pour des applications temps réel sur des appareils sans accélérateur matériel.
4. **Biais potentiel du jeu de données** : L'entraînement sur BSD500 (scènes naturelles généralistes) peut ne pas généraliser à des domaines spécialisés comme l'imagerie médicale, satellite ou industrielle sans ré-entraînement ou adaptation.

5.3 Perspectives pour des travaux futurs

Ces limitations ouvrent la voie à plusieurs perspectives de recherche concrètes, que nous aimerions explorer :

1. **Architectures plus avancées : SwinIR et Transformers** : Une perspective naturelle serait de remplacer le DnCNN par un modèle basé sur les Transformers,

comme le SwinIR [9]. Le mécanisme d'attention pourrait mieux capturer les dépendances à longue portée et reconstruire les textures périodiques. L'implémentation de SwinIR et sa comparaison avec DnCNN sur notre benchmark serait une extension directe et intéressante.

2. **Denoising aveugle (Blind Denoising)** : Développer un modèle capable de débruitter une image sans connaître le niveau de bruit σ a priori. Une piste prometteuse serait d'adapter FFDNet [5], qui prend une carte de bruit estimée en entrée, ou d'ajouter une tête de sortie pour estimer σ simultanément. Cela rendrait le modèle bien plus pratique pour des conditions réelles où le bruit est inconnu.
3. **Adaptation au bruit réel** : Entraîner et évaluer le modèle sur des jeux de données contenant du bruit réel de capteur, comme le dataset SIDD (Smartphone Image Denoising Dataset) ou DND (Darmstadt Noise Dataset). Cela nécessiterait probablement des techniques d'augmentation de données plus sophistiquées et potentiellement un pré-entraînement sur du bruit synthétique suivi d'un fine-tuning sur du bruit réel.
4. **Approches génératives** : Explorer l'utilisation de modèles génératifs comme les GANs (Generative Adversarial Networks) avec une perte adversariale pour favoriser la préservation des textures, ou les modèles de diffusion (Denoising Diffusion Probabilistic Models) qui ont montré des résultats impressionnants en restauration d'image.
5. **Optimisation pour l'embarqué** : Appliquer des techniques de compression de modèles (quantification en int8, pruning, distillation de connaissances) pour réduire la latence d'inférence sur CPU et permettre un déploiement sur des appareils mobiles ou embarqués (edge computing).

Références bibliographiques

- [1] K. Zhang, W. Zuo, Y. Chen, D. Meng, and L. Zhang, "Beyond a Gaussian Denoiser : Residual Learning of Deep CNN for Image Denoising," *IEEE Transactions on Image Processing*, vol. 26, no. 7, pp. 3142-3155, 2017.
- [2] T. S. Huang, G. J. Yang, and G. Y. Tang, "A fast two-dimensional median filtering algorithm," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 27, no. 1, pp. 13-18, 1979.
- [3] C. Tomasi and R. Manduchi, "Bilateral filtering for gray and color images," in *Proceedings of the Sixth International Conference on Computer Vision (ICCV)*, 1998, pp. 839-846.

- [4] H. C. Burger, C. J. Schuler, and S. Harmeling, "Image denoising : Can plain neural networks compete with BM3D?," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2012, pp. 2392-2399.
- [5] K. Zhang, W. Zuo, and L. Zhang, "FFDNet : Toward a Fast and Flexible Solution for CNN-Based Image Denoising," *IEEE Transactions on Image Processing*, vol. 27, no. 9, pp. 4608-4622, 2018.
- [6] O. Ronneberger, P. Fischer, and T. Brox, "U-Net : Convolutional Networks for Biomedical Image Segmentation," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015, pp. 234-241.
- [7] A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NIPS)*, 2017, pp. 5998-6008.
- [8] A. Dosovitskiy et al., "An Image is Worth 16x16 Words : Transformers for Image Recognition at Scale," in *International Conference on Learning Representations (ICLR)*, 2021.
- [9] J. Liang, J. Cao, G. Sun, K. Zhang, L. Van Gool, and R. Timofte, "SwinIR : Image Restoration Using Swin Transformer," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops*, 2021, pp. 1833-1844.
- [10] P. Arbelaez, M. Maire, C. Fowlkes, and J. Malik, "Contour Detection and Hierarchical Image Segmentation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 5, pp. 898-916, 2011.
- [11] K. Dabov, A. Foi, V. Katkovnik, and K. Egiazarian, "Image Denoising by Sparse 3-D Transform-Domain Collaborative Filtering," *IEEE Transactions on Image Processing*, vol. 16, no. 8, pp. 2080-2095, 2007.

A Annexes

A.1 Annexe A : Résumé complet du modèle DnCNN

Ci-dessous le résumé complet de notre modèle implémenté sous Keras. Le nombre total de paramètres est de 211 923, dont 210 963 paramètres entraînaibles (les poids et biais des couches convolutives) et 960 paramètres non entraînaibles (les moyennes et variances de la normalisation par lots).

1	Model : "DnCNN"	
2	-----	
3	Layer (type)	Output Shape Param #
4	=====	
5	input_layer (InputLayer)	[(None, 128, 128, 3)] 0

6	-----		
7	conv2d (Conv2D)	(None, 128, 128, 64)	1792
8	-----		
9	activation (Activation)	(None, 128, 128, 64)	0
10	-----		
11	conv2d_1 (Conv2D)	(None, 128, 128, 64)	36928
12	-----		
13	batch_normalization (BatchNo	(None, 128, 128, 64)	256
14	-----		
15	activation_1 (Activation)	(None, 128, 128, 64)	0
16	-----		
17	conv2d_2 (Conv2D)	(None, 128, 128, 64)	36928
18	-----		
19	batch_normalization_1 (Batch	(None, 128, 128, 64)	256
20	-----		
21	activation_2 (Activation)	(None, 128, 128, 64)	0
22	[-- 15 blocs identiques de Conv+BN+ReLU --]		
23	-----		
24	conv2d_17 (Conv2D)	(None, 128, 128, 64)	36928
25	-----		
26	batch_normalization_16 (Batc	(None, 128, 128, 64)	256
27	-----		
28	activation_17 (Activation)	(None, 128, 128, 64)	0
29	-----		
30	conv2d_18 (Conv2D)	(None, 128, 128, 3)	1731
31	-----		
32	subtract (Subtract)	(None, 128, 128, 3)	0
33	=====		
34	Total params: 211,923		
35	Trainable params: 210,963		
36	Non-trainable params: 960		
37	-----		

Listing 1 – Résumé complet du modèle DnCNN sous Keras

A.2 Annexe B : Exemples de prédictions supplémentaires

Cette annexe présente deux exemples supplémentaires de prédictions du modèle sur l'ensemble de test : (a) un cas où le modèle a très bien performé (PSNR > 40 dB) et (b) un cas plus difficile (PSNR < 32 dB).

TABLE 5 – Exemples de prédictions sur l'ensemble de test.

Image originale	Image bruitée
Originale 1 (ciel, mer)	Bruitée 1
Excellent cas : zones uniformes (ciel, eau) très bonne reconstruction.	
Originale 2 (tapis texture)	Bruitée 2
Cas difficile : texture fine et répétitive (tapis) bruit résiduel visible et léger lissage.	

A.3 Annexe C : Extrait de code clé (entraînement, données, modèle)

Cette section présente les extraits de code les plus importants pour la reproductibilité. Le code complet est disponible sur le dépôt GitHub : https://github.com/SIHAMCODE13/projet_deep_learning.git.

```

1 import tensorflow as tf
2 import numpy as np
3
4 def add_gaussian_noise(images, sigma=25/255.0, seed=None):
5     """Ajoute du bruit gaussien additif aux images."""
6     if seed is not None:
7         tf.random.set_seed(seed)
8     noise = tf.random.normal(shape=tf.shape(images), mean=0.0,
9                             stddev=sigma)
10    return tf.clip_by_value(images + noise, 0.0, 1.0) # Clip pour
11    rester dans [0,1]
12
13 def preprocess_data(images_paths, target_size=(128, 128)):
14     """Charge, redimensionne et normalise les images."""
15     # ... (code de chargement et redimensionnement)
16     return normalized_images # valeurs dans [0,1]

```

Listing 2 – Extrait 1 : Génération du bruit à la volée et préparation des données

```

1 def dncnn_model(input_shape=(128, 128, 3), depth=17, filters=64):
2     """Crée le modèle DnCNN selon l'architecture d'origine."""
3     inputs = tf.keras.Input(shape=input_shape)
4
5     # Première couche
6     x = tf.keras.layers.Conv2D(filters, (3, 3), padding='same')(
7         inputs)

```

```

7     x = tf.keras.layers.Activation('relu')(x)
8
9     # Couches cachées
10    for _ in range(depth - 2):
11        x = tf.keras.layers.Conv2D(filters, (3, 3), padding='same')(x)
12        x = tf.keras.layers.BatchNormalization(axis=-1)(x)
13        x = tf.keras.layers.Activation('relu')(x)
14
15    # Dernière couche
16    residual = tf.keras.layers.Conv2D(input_shape[-1], (3, 3),
17                                       padding='same')(x)
18
19    # Soustraction résiduelle
20    outputs = tf.keras.layers.Subtract()([inputs, residual])
21
22    model = tf.keras.Model(inputs=inputs, outputs=outputs)
23    return model

```

Listing 3 – Extrait 2 : Définition de l'architecture DnCNN

```

1  import tensorflow as tf
2  from tensorflow.keras.callbacks import EarlyStopping,
3      ReduceLROnPlateau
4
5  # Fixer les graines aléatoires
6  tf.random.set_seed(42)
7  np.random.seed(42)
8
9  # Compilation
10 model = dncnn_model()
11 model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e
12             -3), loss='mse')
13
14 # Callbacks
15 callbacks = [
16     ReduceLROnPlateau(monitor='val_loss', patience=5, factor=0.5,
17                       min_lr=1e-6, verbose=1),
18     EarlyStopping(monitor='val_loss', patience=10,
19                  restore_best_weights=True, verbose=1)
19 ]

```

```
18 # Entraînement
19 history = model.fit(
20     x_train_noisy, x_train_clean,
21     validation_data=(x_val_noisy, x_val_clean),
22     batch_size=32,
23     epochs=50,
24     callbacks=callbacks,
25     verbose=1
26 )
```

Listing 4 – Extrait 3 : Entraînement avec callbacks et seed fixée

A.4 Annexe D : Environnement technique complet

L’environnement d’exécution complet (fichier requirements.txt) est le suivant :

```
1 python==3.10.12
2 tensorflow-gpu==2.19.0
3 numpy==1.26.4
4 matplotlib==3.8.0
5 scikit-learn==1.5.1
6 opencv-python==4.10.0.84
7 pillow==10.3.0
8 scipy==1.13.0
9 tqdm==4.66.4
```

Listing 5 – requirements.txt

A.5 Annexe E : Lien GitHub et instructions d’exécution

Le projet est disponible publiquement sur GitHub à l’adresse suivante :

https://github.com/SIHAMCODE13/projet_deep_learning.git

Pour reproduire les expériences :

1. Cloner le dépôt : `git clone https://github.com/SIHAMCODE13/projet_deep_learning.git`
2. Installer les dépendances : `pip install -r requirements.txt`
3. Lancer le notebook principal : `jupyter notebook DnCNN_denoising.ipynb`
4. Toutes les graines aléatoires sont fixées (seed=42), les résultats doivent être reproductibles.